SOFTWARE ENGINEERING – PROJECT PROPOSAL

PrivateFind

AI-assisted campus lost and found with private ownership verification and a smart locker

Course	Software Engineering, Sejong University
Instructor	Rajendra Dhakal, Ph.D.
Team	Roaster
Members	Zohidjon Mahmudjonov Murodjon Ugli, Umarov Muslimbek, Gerthofer Simon, Falk Raphael
Category	Web application with embedded / IoT component
Version	Week 5 (1 October 2026): system design added (chapter 9)

Contents

1 Project Title and Team Roster 3

2 System Architecture 3

2.1 System Overview 4

3 Tools Used and Technology Stack 5

4 Description and Working Principle 5

4.1 Problem 5

4.2 Main Functions 5

4.3 Matching 5

4.4 Ownership Verification 6

4.5 Smart Locker and Boundary Conditions 6

4.6 Requirements (SRS Summary) 6

5 Work Distribution 6

5.1 Work Breakdown Structure 7

5.2 Effort Estimation with COCOMO 7

6 Detailed Timeline Weeks 1 to 13 8

6.1 Gantt Chart 8

6.2 Staff Allocation 11

6.3 Critical Path 11

7 Work Completed Before Week 3 11

7.1 Team Formation and Skill Matrix 11

7.2 Ideation and Theme Selection 11

7.3 Problem Statement, Personas and Scope 11

7.4 Initial System Architecture Sketch 12

8 Development Process and Main Risks 12

9 System Design 12

9.1 Use Case Diagram 13

9.2 Sequence Diagram: Collect Item 13

9.3 State Diagram: Locker Compartment 14

9.4 Database Design 15

9.5 Interfaces 16

9.6 User Interface 16

9.7 Locker Hardware 16

10 References 18

1 Project Title and Team Roster

Project title: PrivateFind – an AI-assisted campus lost-and-found system with private ownership verification and smart locker handover.

A finder puts a found item into a smart locker and uploads a few photos. AI turns the photos into item attributes, and a Match Score suggests possible items to the owner. Before the locker opens, the owner has to prove ownership with private details.

Name	Student ID	Role	E-mail
Zohidjon Mahmudjonov Murodjon Ugli	23013058	Team leader, AI and backend	zohidjon.mah@gmail.com
Umarov Muslimbek	23012870	UI and frontend	muslimbekumarov8@gmail.com
Gerthofer Simon	26040296	Database and data layer	simon.gerthofer@gmail.com
Falk Raphael	26040280	IoT, hardware and locker	1401raphael@gmail.com

We work in one GitHub repository and meet once a week. Zohidjon leads the meeting and checks every submission.

2 System Architecture

PrivateFind follows the pipeline required for embedded projects: sensors □ ESP32 □ REST API □ database □ web UI (Figure 1).

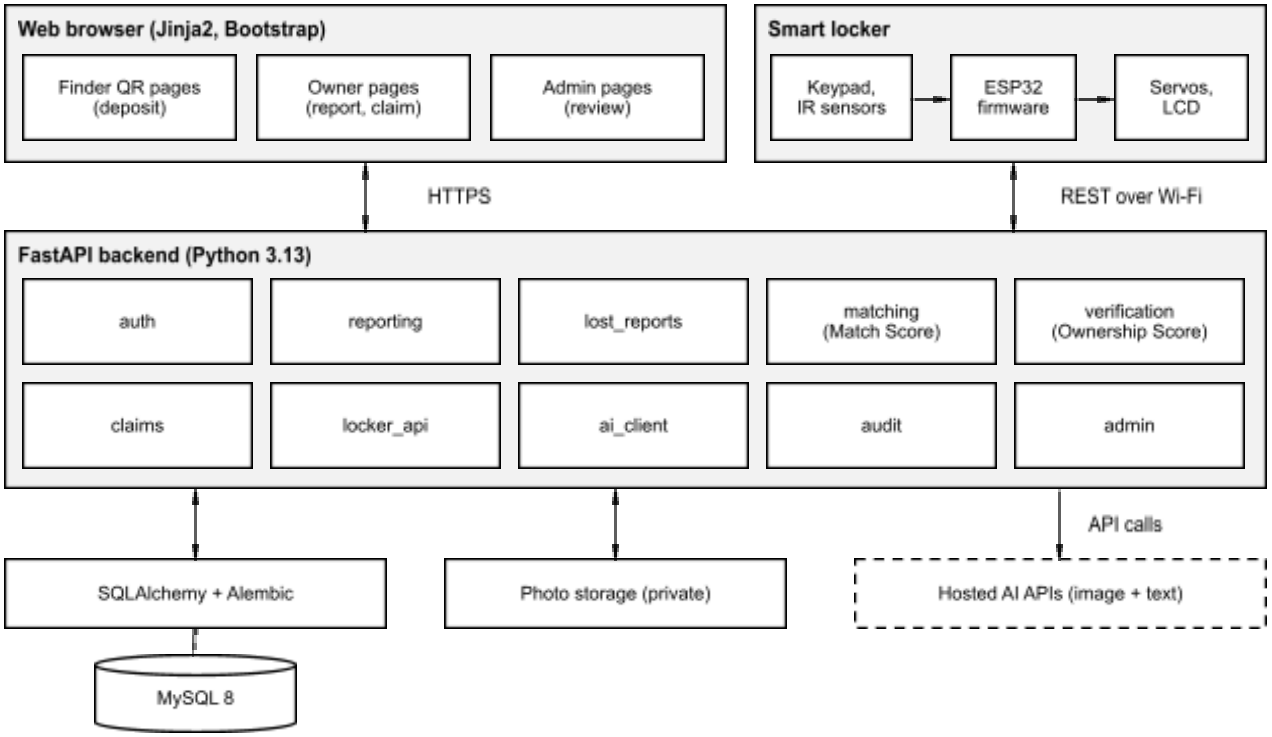


Figure 1: System architecture and backend modules

Figure 2 on the next page shows the whole system on one page: the users, the workflow steps 1–6 and who builds which part.

2.1 System Overview

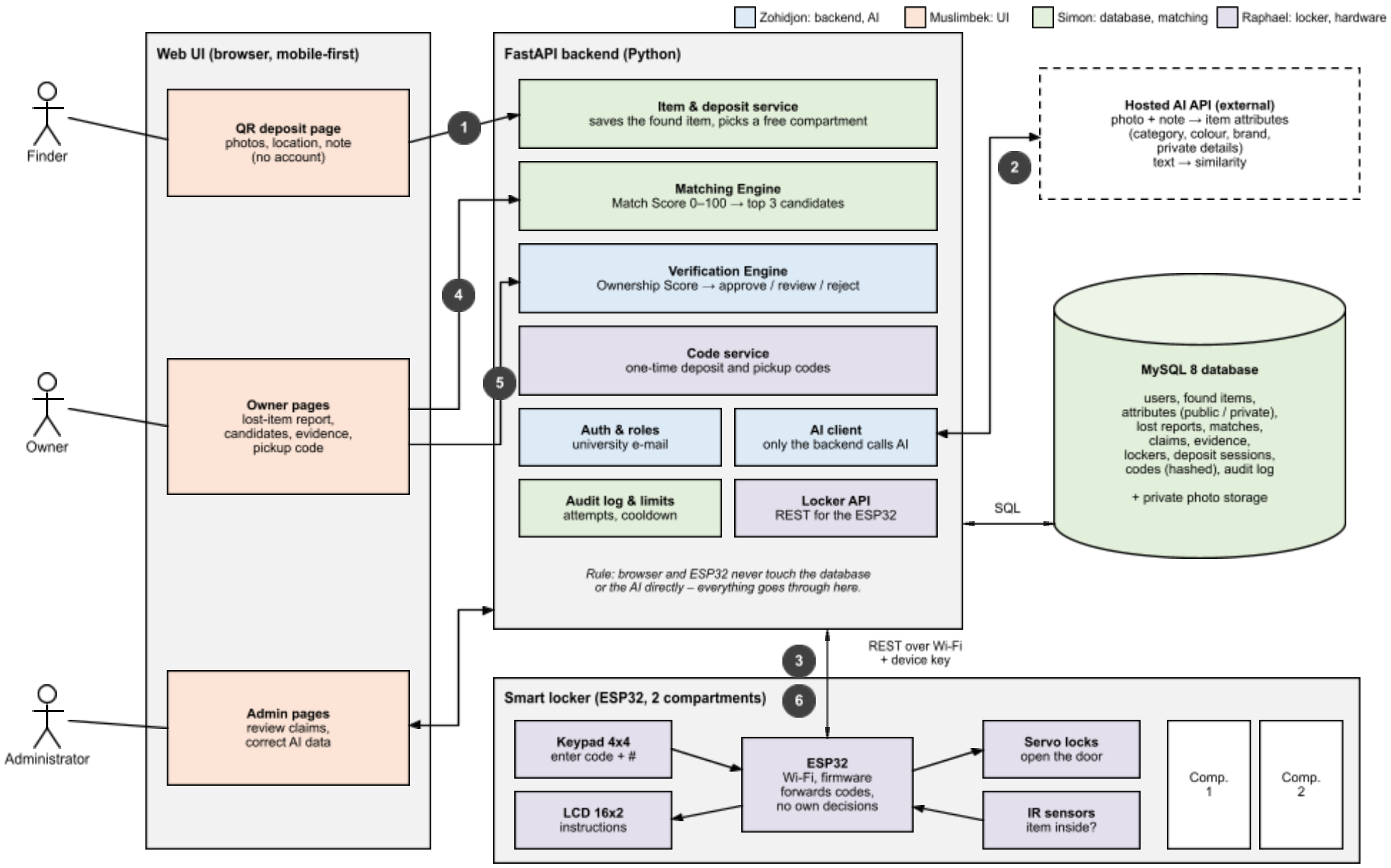


Figure 2: System overview with users, workflow steps 1–6 and team responsibilities

Four rules keep the parts separate:

- The browser and the ESP32 only talk to the FastAPI backend.
- Only the backend accesses MySQL (through SQLAlchemy).
- Only the backend calls the AI APIs. The API key stays on the server.
- The ESP32 does not decide if a code is valid. It only forwards the code.

Runtime data flow (numbers as in Figure 2)

1. The finder scans the QR code on the locker and uploads 1–3 photos, the location and a short note.
2. The AI extracts attributes. The backend assigns a free compartment and a deposit code.
3. The finder enters the code, puts the item in and closes the door. The IR sensor confirms it.
4. The owner signs in, reports the lost item and sees the top candidates.
5. The owner submits private details. The system approves, rejects or sends the claim to an admin.
6. After approval the owner gets a one-time pickup code and opens the compartment.

3 Tools Used and Technology Stack

Area	Tools
Backend	Python 3.13, FastAPI, Pydantic
Database	MySQL 8, SQLAlchemy 2, Alembic
Frontend	HTML, Jinja2, Bootstrap 5, JavaScript
AI	Hosted multimodal API (photo and text), text embeddings; no own model training
Embedded	ESP32, Arduino C++
Security	Sessions or JWT, Argon2 or BCrypt
Testing	pytest, HTTPX
Deployment	Docker and Docker Compose (backend, MySQL and web UI in containers)
Design and version control	Figma, Git and GitHub

Hardware

Component	Qty	Status
ESP32 DevKit (ESP32-WROOM-32D)	1	Bought in Week 5
4x4 keypad	1	Bought in Week 5
SG90 servo (door lock)	2	1 bought, 1 in Week 6
IR presence sensor	2	Week 6
16x2 LCD with I2C module	1	Week 6
Breadboards, jumper wires, capacitor	–	Bought in Week 5
Enclosure with 2 compartments	1	Weeks 9–11

4 Description and Working Principle

4.1 Problem

Finders do not want to spend time on a long registration. Public photos help people who claim items that are not theirs. And a good AI match is not a proof of ownership. PrivateFind keeps the deposit quick, keeps the evidence private and separates matching from verification.

4.2 Main Functions

- Quick deposit with a QR code, no account for the finder
- AI extraction of public and private item attributes
- Lost-item report by the owner before any candidate is shown
- Match Score and the top three candidates
- Ownership claim with private evidence, admin review for unclear cases
- One-time deposit and pickup codes at the locker
- Audit log of all important actions

4.3 Matching

Open items are filtered by category and date. The Match Score (0–100) is calculated as:

25% category + 15% location + 15% time + 25% attributes + 20% semantic similarity

Up to three candidates with a score of at least 60 are shown. The score only ranks candidates; it never proves ownership.

4.4 Ownership Verification

The claimant describes private details, for example the contents, a sticker, damage or the last digits of a serial number. Each detail has a weight (strong 3, distinctive 2, generic 1). The Ownership Score decides:

- 80 or more and no critical conflict: approved, pickup code is issued
- 50–79 or a generic item (e.g. a plain umbrella): review by an administrator
- Below 50, critical mismatch or more than 3 failed attempts: rejected, cooldown

The claimant is never told which detail was wrong. The first approved claim reserves the item.

4.5 Smart Locker and Boundary Conditions

The locker has two compartments with opaque doors. Each has a servo lock and an IR sensor. Deposit and pickup codes are single-use and belong to one compartment.

- Owners need a university e-mail account. Finders need no account.
- Expired or used codes never open the locker.
- If no compartment is free, the QR page does not accept a deposit.
- If the AI or the locker is not available, we use mock services. Failed AI jobs are retried or corrected by an admin.

4.6 Requirements (SRS Summary)

The SRS was finished in Week 3. The table shows the main functional (FR) and non-functional (NFR) requirements.

ID	Requirement
FR1	The finder deposits an item with the QR code, 1–3 photos, location and a note, without an account.
FR2	The system extracts public and private attributes from the photos with a hosted AI.
FR3	Owners sign in with a university e-mail and create a lost-item report.
FR4	The system calculates a Match Score and shows the top 3 candidates with public data only.
FR5	The owner submits private evidence. The system approves, rejects or sends the claim to an admin.
FR6	The locker opens only with a valid one-time deposit or pickup code.
FR7	Administrators review claims, correct item data and see the audit log.
NFR1	Security: photos and private attributes are never public. Passwords and codes are stored as hashes.
NFR2	Only the backend uses the database and the AI. The AI never approves claims or opens the locker.
NFR3	Performance: web requests take under 2 seconds, AI processing under 10 seconds in the demo.
NFR4	Usability: a deposit takes less than one minute on a phone.
NFR5	Reliability: every code check and claim decision is logged. Max. 3 failed attempts per claim.

Data storage: all data is stored in MySQL, photos in a private folder on the server. UI scope: mobile-first web pages for three roles (finder, owner, administrator).

5 Work Distribution

Module / deliverable	Owner	Support
Requirements (SRS)	All	Zohidjon consolidates
Database model and migrations	Simon	Zohidjon
Matching Engine (Match Score)	Simon	Zohidjon (AI similarity)
AI attribute extraction	Zohidjon	Muslimbek (upload UI)
Authentication and roles	Zohidjon	Simon
Ownership Verification Engine	Zohidjon	Simon
Audit log, attempt and rate limits	Simon	Zohidjon
Wireframes and all UI screens	Muslimbek	Zohidjon, Simon
Locker REST API and code lifecycle	Raphael	Zohidjon
ESP32 firmware and physical locker	Raphael	Simon
Testing and final QA	Zohidjon	All

5.1 Work Breakdown Structure

The WBS has 12 phases and 54 work packages with 455 planned hours (Figure 3). All work packages with their hours are listed in the Gantt chart (Figure 4).

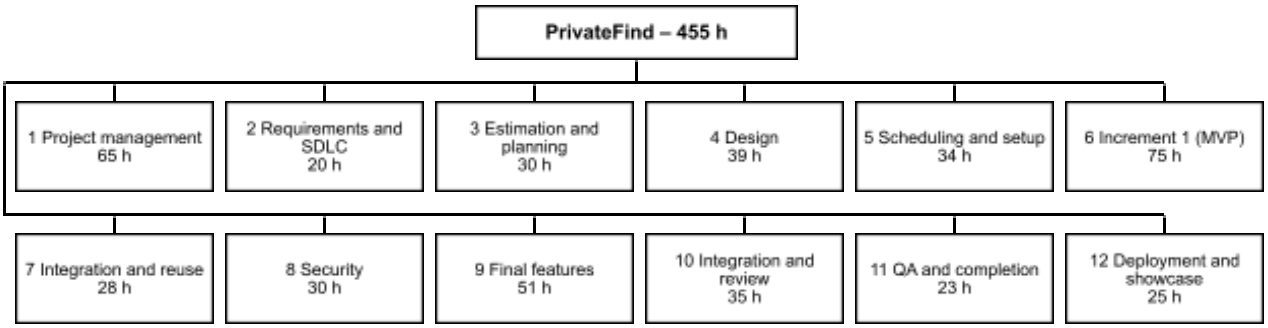


Figure 3: Work breakdown structure (level 1) with planned hours

Member	Hours	Share
Zohidjon	134	29 %
Muslimbek	100	22 %
Simon	119	26 %
Raphael	102	22 %
Total	**455**	**100 %**

5.2 Effort Estimation with COCOMO

We estimated the size of each module with three values (optimistic, likely, pessimistic) and the PERT formula (O + 4M + P) / 6. Only our own code is counted, not tests or reused libraries.

Module	Language	O	M	P	Expected LOC
Authentication and users	Python	250	400	600	408
Found-item reporting and QR deposit	Python	300	450	700	467
AI service integration	Python	250	400	650	417
Matching Engine	Python	250	400	600	408
Ownership Verification Engine	Python	300	450	700	467
Claims, reservation, admin review	Python	250	400	600	408
Locker REST API and codes	Python	200	350	550	358
Security: limits and audit log	Python	200	300	500	317
Data layer (models, migrations)	Python/SQL	400	600	900	617
Web UI (templates, JavaScript)	HTML/JS	800	1,200	1,800	1,233
ESP32 firmware	C++	350	550	850	567
Total		**3,550**	**5,500**	**8,450**	**5,667**

With 5.67 KLOC we used Basic COCOMO (Boehm, 1981): E = a × KLOC^b, D = 2.5 × E^d, Staff = E / D. One person-month (PM) is 152 working hours.

Mode	a / b	Effort (PM)	Duration (months)	Staff	Effort (h)
Organic	2.4 / 1.05	14.8	7.0	2.1	2,254
Semi-detached (used)	**3.0 / 1.12**	**20.9**	**7.2**	**2.9**	**3,182**
Embedded	3.6 / 1.20	28.9	7.3	3.9	4,387

We use semi-detached mode, because our experience is mixed and the web, AI and embedded parts work together. An Intermediate COCOMO check with 15 cost drivers (EAF 1.46, mainly because of the short schedule) gives 30.5 PM. Our plan has 420 hours for Weeks 3–13 (455 hours with Weeks 14–15). COCOMO assumes professional projects without much reuse. We reuse frameworks and a hosted AI, so we see the gap as a scope risk: strict scope, mocks and a new estimate after the MVP.

6 Detailed Timeline Weeks 1 to 13

Week	Phase	Work and deliverable (milestone)
1	Ideation	Team, topic, first proposal
2	Requirements	SRS draft
3	SDLC	Final SRS and SDLC justification (M1)
4	Estimation	COCOMO, WBS, Gantt chart (M2)
5	Design	System design: architecture, ER diagram, UI mockups (M3)
6	Setup	Final Gantt chart, repository, DB schema, FastAPI skeleton (M4)
7	Core logic	QR deposit, lost report, login, DB persistence (M5); proposal presentation on 13 and 15 Oct
8	MVP	Midterm demo with a mock locker (M6)
9	Reuse	Libraries, AI API, embeddings, ESP32 libraries
10	Security	Access control, input validation, attempt and rate limits
11	Final features	Verification, admin review, physical locker (M7)
12	Integration	End-to-end tests, code review, release candidate (M8)
13	Testing	Unit, API, DB and locker tests; final code and test report (M9)
14	Deployment	Setup scripts, user and maintenance manual
15	Showcase	Live demo and final presentation (M10)

Milestones and definition of done:

ID	Milestone	Week	Definition of done
M1	Requirements and SDLC approved	3	Team agrees on finder effort, AI limits, verification rules and scope.
M2	Estimation, WBS and schedule agreed	4	COCOMO, WBS and first Gantt chart with staff allocation are agreed.
M3	Design approved	5	Architecture, ER diagram, AI data format and wireframes are reviewed.
M4	Schedule, repository and DB ready	6	Final schedule; repository, dev environment and DB schema exist.
M5	First running increment	7	QR deposit, lost report and DB persistence work through FastAPI.
M6	MVP	8	A photo becomes item attributes; one candidate and claim work end to end.
M7	Feature complete	11	Matching, verification, admin review and the real locker work together.
M8	Release candidate	12	All must-have features are integrated, tested and reviewed.
M9	Project complete	13	All tests are done and no high-severity bug is open.
M10	Final showcase	15	Live demo and final presentation are given.

6.1 Gantt Chart

Figure 4 shows two screenshots of the sheet “Gantt Chart” from our planning workbook. Each bar is one work package with its owner and hours. Columns for status and progress tracking are hidden.

PrivateFind – Project Schedule (Gantt Chart), Weeks 3–15

Team Roaster · Software Engineering · Sejong University · Instructor: Rajendra Dhakal, Ph.D.

ID	Task / work package	Owner	Support	Start week	End week	Dura-tion (wks)	Effort total (h)	Zohid-jon (h)	Muslim-bek (h)	Simon (h)	Raphael (h)	Deliverable
1	**Project management**			**3**	**15**	**13**	**65**	**26**	**13**	**13**	**13**	
1.1	Weekly team meeting, coordination, schedule review & submission review	Zohidjon	All	3	15	13	65	26	13	13	13	Weekly deliverables
2	**Requirements & SDLC**			**3**	**3**	**1**	**20**	**9**	**3**	**5**	**3**	
2.1	Finalize SRS (functional & non-functional requirements)	Zohidjon	All	3	3	1	15	6	3	3	3	Finalized SRS
2.2	SDLC selection & justification (incremental model)	Zohidjon	Simon	3	3	1	5	3	–	2	–	SDLC justification
M1	*Requirements & SDLC approved*	*All*		*3*	*3*							
3	**Estimation & planning**			**4**	**7**	**4**	**30**	**9**	**3**	**13**	**5**	
3.1	COCOMO estimation (KLOC, effort, duration)	Simon	Zohidjon	4	4	1	7	2	–	5	–	COCOMO report
3.2	Work breakdown structure (WBS)	Zohidjon	All	4	4	1	6	3	1	1	1	WBS
3.3	Initial project schedule (Gantt chart) & staff allocation	Simon	Zohidjon	4	4	1	6	1	–	5	–	Initial project schedule
3.4	Order hardware (ESP32, IR sensors, keypad, LCD, servos)	Raphael		4	4	1	2	–	–	–	2	MVP hardware set (bought 30 Sep)
M2	*Estimation, WBS & initial schedule agreed*	*All*		*4*	*4*							
3.5	Project proposal presentation (walk-through of the proposal document)	Zohidjon	All	6	7	2	9	3	2	2	2	Proposal presentation (13 & 15 Oct)
4	**Design**			**4**	**5**	**2**	**39**	**10**	**12**	**9**	**8**	
4.1	System architecture & module diagram	Zohidjon	Raphael	5	5	1	4	3	–	–	1	System design spec
4.2	REST API contracts (web + locker)	Zohidjon	Raphael	5	5	1	4	2	–	–	2	System design spec
4.3	AI input/output schema (item attributes)	Zohidjon	Simon	5	5	1	4	3	–	1	–	System design spec
4.4	ER diagram & database design	Simon	Zohidjon	5	5	1	7	1	–	6	–	ERD
4.5	Figma wireframes & user flows (finder, owner, admin)	Muslimbek	All	4	5	2	15	1	12	1	1	UI mockups
4.6	Locker hardware design (wiring, 2 compartments)	Raphael	Simon	5	5	1	5	–	–	1	4	System design spec
M3	*Design approved (UI mockups + ERD)*	*All*		*5*	*5*							
5	**Scheduling & setup**			**5**	**6**	**2**	**34**	**9**	**7**	**9**	**9**	
5.1	Git repository, branching rules & dev environment	Zohidjon	All	6	6	1	6	3	1	1	1	Repo URL
5.2	MySQL schema, SQLAlchemy models & Alembic migrations	Simon	Zohidjon	6	6	1	9	1	–	8	–	DB schema
5.3	FastAPI skeleton & AI service mock	Zohidjon		6	6	1	5	5	–	–	–	
5.4	Frontend base templates (Jinja2, Bootstrap)	Muslimbek		6	6	1	6	–	6	–	–	
5.5	ESP32 breadboard test (keypad, LCD, IR sensor, servo)	Raphael		5	6	2	8	–	–	–	8	
M4	*Schedule, repository & database baseline ready*	*All*		*6*	*6*							
6	**Increment 1: core logic & MVP**			**7**	**8**	**2**	**75**	**20**	**21**	**18**	**16**	
6.1	Authentication & roles (university e-mail)	Zohidjon	Simon	7	7	1	8	6	–	2	–	
6.2	QR finder flow & photo upload	Muslimbek	Zohidjon	7	7	1	8	1	7	–	–	
6.3	Item persistence & private photo storage	Simon	Zohidjon	7	7	1	7	1	–	6	–	
6.4	Lost-item report form	Muslimbek	Simon	7	7	1	6	–	5	1	–	
6.5	AI attribute extraction (real API call)	Zohidjon	Muslimbek	7	8	2	9	8	1	–	–	
6.6	Match Score v1 (category, location, time, attributes)	Simon		7	8	2	9	–	–	9	–	
6.7	Locker REST endpoints & deposit/pickup code lifecycle (mock locker)	Raphael	Zohidjon	7	8	2	10	2	–	–	8	
6.8	ESP32 firmware v1 (Wi-Fi, keypad → REST)	Raphael		7	8	2	8	–	–	–	8	
6.9	Candidate results & claim submission screens	Muslimbek	Zohidjon	8	8	1	10	2	8	–	–	
M5	*First running increment (working DB connectivity)*	*All*		*7*	*7*							
M6	*MVP / midterm demo (mock locker)*	*All*		*8*	*8*							
7	**Integration & reuse**			**9**	**10**	**2**	**28**	**7**	**9**	**6**	**6**	
7.1	Integrate reused libraries (Bootstrap components, FastAPI libs, AI SDK)	Zohidjon	Muslimbek	9	9	1	7	4	3	–	–	Codebase update
7.2	Semantic similarity (embeddings) in Match Score	Simon	Zohidjon	9	9	1	8	2	–	6	–	Codebase update
7.3	ESP32 peripheral libraries & final locker REST contract	Raphael	Zohidjon	9	9	1	7	1	–	–	6	Codebase update



ID	Task / work package	Owner	Support	Start week	End week	Dura-tion (wks)	Effort total (h)	Zohid-jon (h)	Muslim-bek (h)	Simon (h)	Raphael (h)	Deliverable
7.4	UI/UX fixes from midterm feedback	Muslimbek		9	10	2	6	–	6	–	–	
8	**Security implementation**			**10**	**10**	**1**	**30**	**9**	**5**	**10**	**6**	
8.1	Access control & private-photo rules	Zohidjon	Muslimbek	10	10	1	6	5	1	–	–	Security audit
8.2	Input validation, upload checks & SQL injection prevention	Simon	Muslimbek	10	10	1	7	–	3	4	–	Security audit
8.3	Attempt limits, cooldowns, rate limits & code expiry	Simon	Zohidjon	10	10	1	6	1	–	5	–	Security audit
8.4	ESP32 device authentication	Raphael	Zohidjon	10	10	1	6	1	–	–	5	Security audit
8.5	Security implementation audit (document)	Zohidjon	All	10	10	1	5	2	1	1	1	Security audit
9	**Final feature development**			**9**	**11**	**3**	**51**	**15**	**8**	**10**	**18**	
9.1	Ownership Verification Engine (score, critical evidence, thresholds)	Zohidjon	Simon	9	11	3	15	12	–	3	–	Feature-complete code
9.2	Audit log (AI results, claims, locker events)	Simon		11	11	1	5	–	–	5	–	Feature-complete code
9.3	Administrator review screens	Muslimbek	Zohidjon	11	11	1	9	1	8	–	–	Feature-complete code
9.4	Physical locker build (enclosure, 2 compartments, servos, sensors)	Raphael	Simon	9	11	3	14	–	–	2	12	Feature-complete code
9.5	Physical locker ↔ backend integration	Raphael	Zohidjon	11	11	1	8	2	–	–	6	Feature-complete code
M7	*Feature complete*	*All*		*11*	*11*							
10	**Integration & review**			**12**	**12**	**1**	**35**	**9**	**7**	**12**	**7**	
10.1	End-to-end scenarios (deposit → match → claim → pickup)	Zohidjon	All	12	12	1	9	3	2	2	2	RC build
10.2	Score evaluation with synthetic match / non-match cases	Simon	Zohidjon	12	12	1	6	1	–	5	–	RC build
10.3	Peer code reviews (pull requests)	All		12	12	1	8	2	2	2	2	RC build
10.4	Bug fixing & stabilization	All		12	12	1	12	3	3	3	3	RC build
M8	*Release candidate (RC) build*	*All*		*12*	*12*							
11	**Quality assurance & completion**			**13**	**13**	**1**	**23**	**6**	**5**	**7**	**5**	
11.1	Unit, API & AI-service failure/retry tests (pytest, HTTPX)	Zohidjon	Simon	13	13	1	5	4	–	1	–	Test report
11.2	Database & transaction tests (competing claims)	Simon		13	13	1	4	–	–	4	–	Test report
11.3	UI tests (finder, owner, admin flows)	Muslimbek		13	13	1	4	–	4	–	–	Test report
11.4	Locker/device failure tests	Raphael	Simon	13	13	1	5	–	–	1	4	Test report
11.5	Test report	Zohidjon	All	13	13	1	5	2	1	1	1	Test report
M9	*Project complete: final source, executable & test report*	*All*		*13*	*13*							
12	**Deployment & final presentation (post-development)**			**14**	**15**	**2**	**25**	**5**	**7**	**7**	**6**	
12.1	Deployment package & setup scripts	Simon	Zohidjon	14	14	1	6	2	–	4	–	Distribution package
12.2	User & maintenance manual	Muslimbek	Raphael	14	14	1	7	–	4	–	3	Manuals
12.3	Final presentation & live demo preparation	All		14	15	2	12	3	3	3	3	Presentation
M10	*Final capstone showcase*	*All*		*15*	*15*							
	Total planned effort (hours)						**455**	**134**	**100**	**119**	**102**	

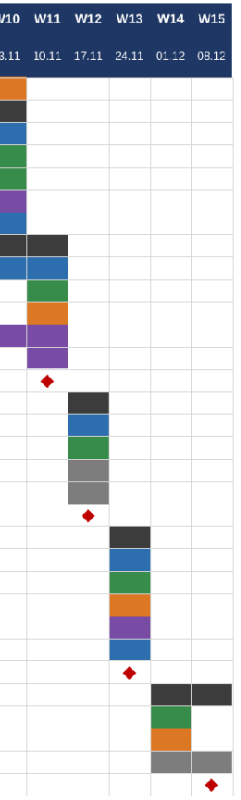


Figure 4: Gantt chart of PrivateFind, Weeks 3 to 15

6.2 Staff Allocation

Planned hours per member and week (rounded):

Member	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	Total
Zohidjon	11	9	12	12	17	9	13	15	9	11	8	5	3	134
Muslimbek	4	8	7	9	15	10	7	9	9	8	6	6	2	100
Simon	6	13	9	11	16	5	9	13	8	13	8	6	2	119
Raphael	4	5	13	7	10	9	11	11	11	8	6	5	2	102
Team	25	35	41	39	58	33	40	48	37	40	28	22	9	**455**

On average each member works about 9 hours per week. Week 7 is the peak (about 16-17 hours for Zohidjon and Simon), because the presentation and the first implementation week come together. Support hours can be moved to Week 6.

6.3 Critical Path

The course has a deliverable almost every week, so nearly all packages are critical. Only 7.1 and 7.2 have one or two weeks of float. The longest chains are:

- Software: ER diagram □ DB schema □ item persistence □ Match Score □ MVP □verification □ feature complete
- Hardware: order parts □ breadboard test □ ESP32 firmware □ locker build □ integration

We check the plan in every weekly meeting and update the workbook if something is late.

7 Work Completed Before Week 3

7.1 Team Formation and Skill Matrix

Member	Frontend	Backend / AI	Database	Hardware
Zohidjon	Working	Primary	Working	Basic
Muslimbek	Primary	Basic	Basic	Basic
Simon	Basic	Working	Primary	Support
Raphael	Basic	Working	Support	Primary

Primary = responsible, Working = can implement, Support = helps, Basic = can test.

7.2 Ideation and Theme Selection

Session	Discussion	Decision
8 Sep	Simple lost-and-found website vs. locker only vs. both. A website alone is too simple, a locker alone shows too little software.	Ownership verification as core problem, locker for the handover
10 Sep	Finders get no benefit, so the effort for them must be very low.	QR code, photo and short note, no finder account
17 Sep	Public photos help false claims; an AI match alone is no proof.	Hosted AI for attributes, Match Score, separate Ownership Score

The scope is feasible: we use an existing AI API instead of training a model, the rules are simple and testable, and the locker has only two compartments.

7.3 Problem Statement, Personas and Scope

A finder needs a fast way to secure an item. An owner needs good matches without showing the details that prove ownership.

Persona	Goal	Needs
Finder	Hand over a found item quickly	QR code, under one minute, no contact data
Owner	Find the item and prove ownership	Good ranking, private evidence, safe pickup
Administrator	Handle unclear claims and AI errors	Private evidence view, audit history

In scope: QR deposit, private photo storage, AI attributes, owner accounts, Match Score, ownership claims, one-time codes, two-compartment locker, audit log, admin review.

Out of scope: model training, public photo browsing, rewards, chatbot, mobile app, connection to university systems.

7.4 Initial System Architecture Sketch

The first sketch already had the boundaries of Figure 1: browser, backend, database and ESP32. Only the backend talks to the database and the AI service.

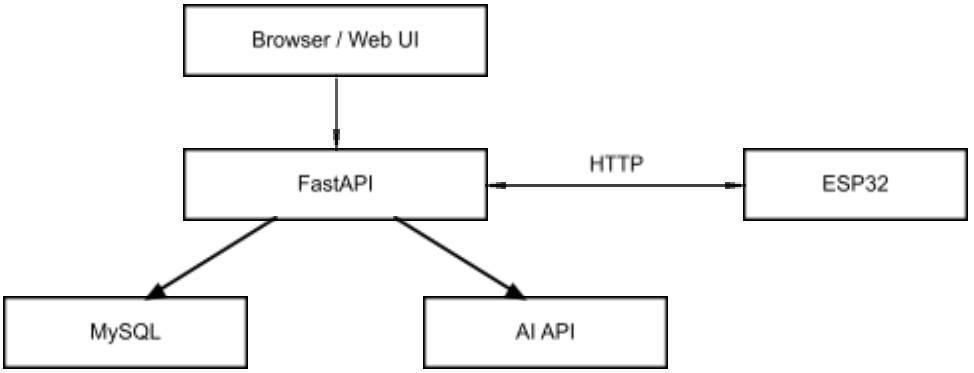


Figure 5: Initial system architecture sketch from the early project phase

8 Development Process and Main Risks

In Week 3 we compared the development models from the lecture and chose incremental development:

Model	Fit for PrivateFind
Waterfall	No. AI and hardware results are uncertain, so we need feedback before the end.
V-Model	No. Strong test planning, but too much documentation for our 13 weeks.
Agile (Scrum, XP)	Partly. We use short cycles and weekly meetings, but not the full Scrum roles.
Incremental (chosen)	Yes. Requirements are fixed early, then we deliver working increments.

Increment 1 (Weeks 7–8) ends with the MVP. Increment 2 (Weeks 9–11) adds reuse, security and the remaining features.

Risk	Impact	Mitigation
AI extracts a wrong attribute	Medium	Store confidence, admin can correct, test with real photos
False claimant	High	Hide photos and details, owner report first, attempt limits
Generic item, weak evidence	Medium	No automatic approval, admin review
AI service slow or unavailable	Medium	Mock in development, retry, limit images
Hardware delay	Medium	Mock locker until Week 9; MVP parts already work (Week 5)
Scope growth	Medium	No chatbot, rewards or mobile app
Campus Wi-Fi blocks the ESP32	Low	Phone hotspot for the demo
Workload peaks	Medium	Weekly Gantt update, move support hours

9 System Design

This chapter is the Week 5 deliverable. Figure 1 in chapter 2 shows the architecture and the modules. Here we add UML models from Chapter 5 for the main interactions and the locker behaviour, the ER diagram and the UI wireframes.

9.1 Use Case Diagram

Actors can be people or other systems. Finder, owner and administrator are people; the smart locker and the AI service are systems (Figure 6).

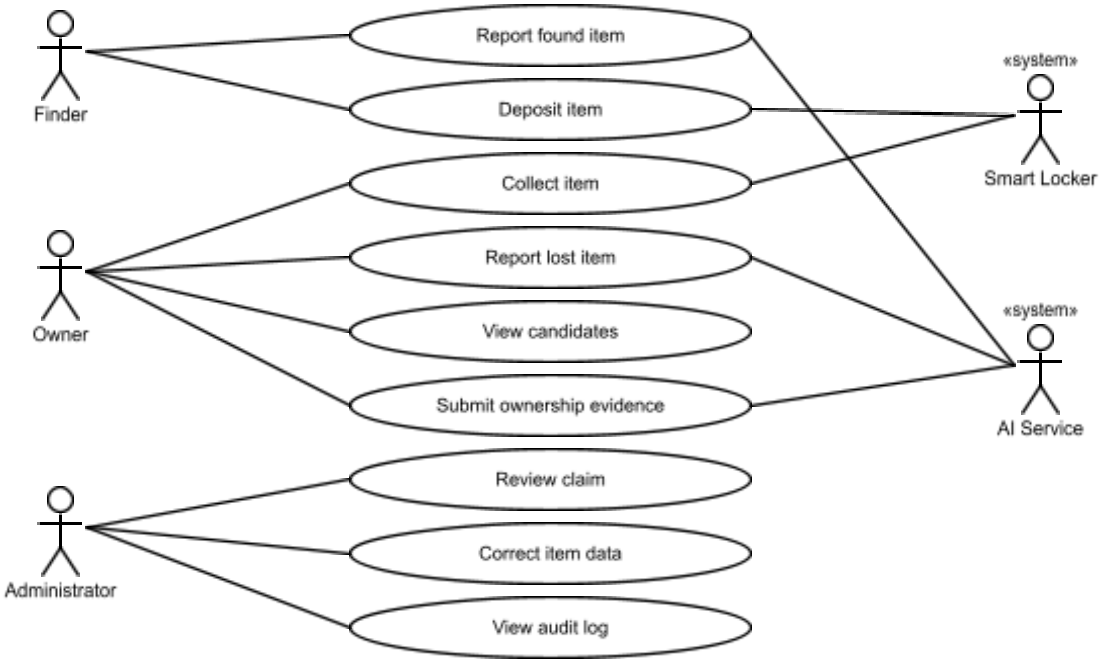


Figure 6: Use case diagram

9.2 Sequence Diagram: Collect Item

Figure 7 shows the pickup at the locker. The ESP32 only forwards the code; the backend checks it and tells the locker which compartment to open.

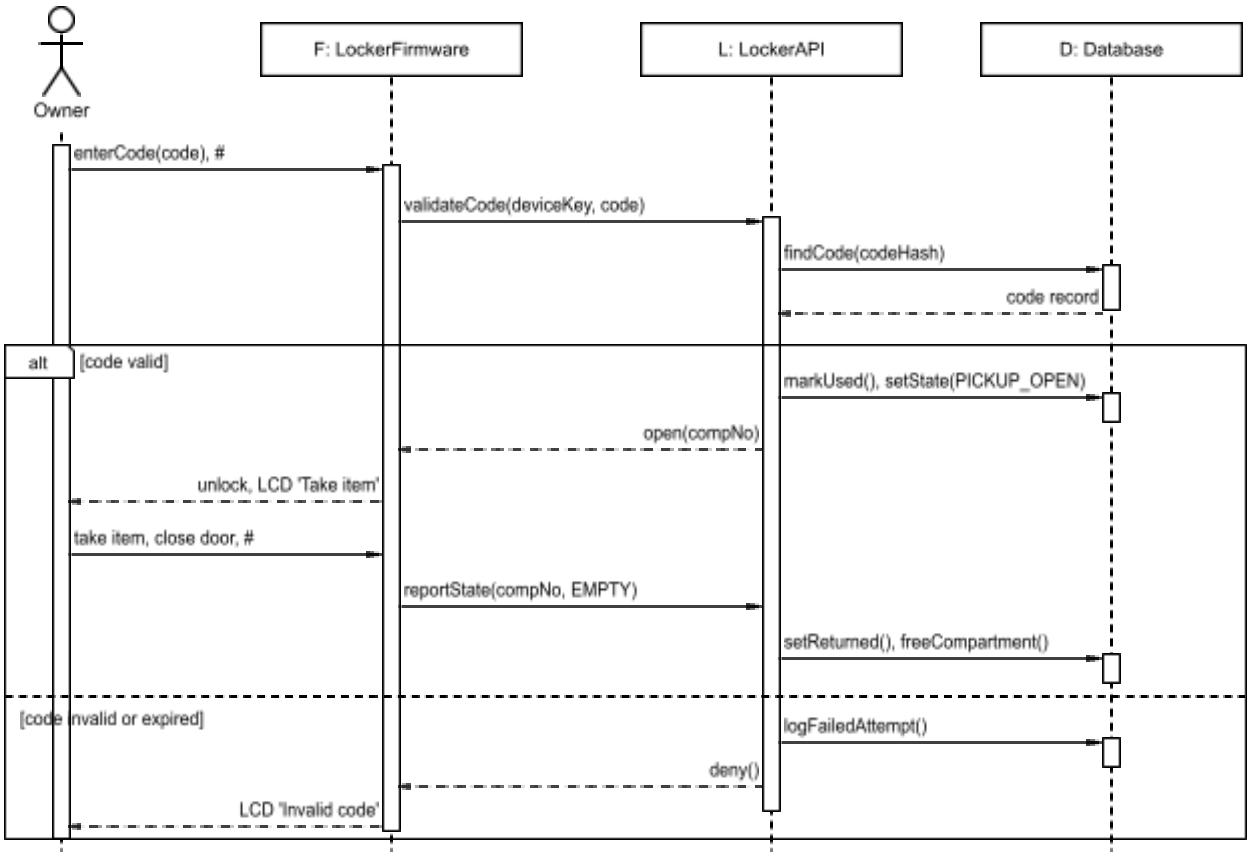


Figure 7: Sequence diagram for “Collect item”

9.3 State Diagram: Locker Compartment

Each compartment is always in one of six states (Figure 8). “Door closed” means the user closed the door and pressed #, because the locker has no door sensor. The IR sensor then shows if an item is inside.

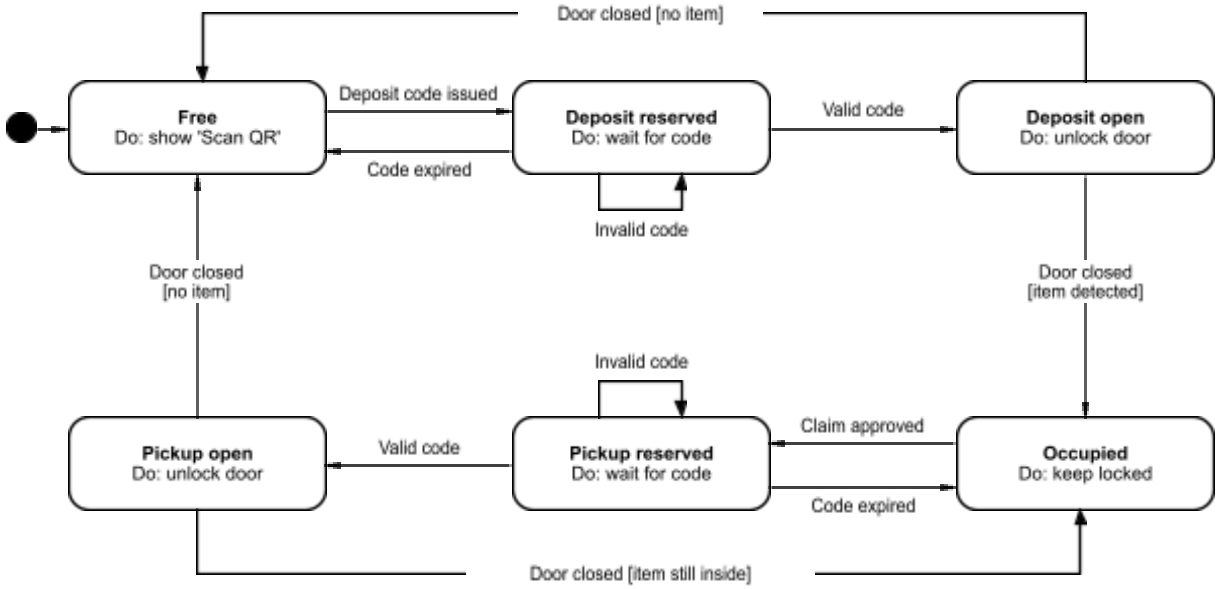


Figure 8: State diagram of a locker compartment

9.4 Database Design

Figure 9 shows the ER diagram with 13 tables. We use the UML multiplicities from Chapter 5: 1 = exactly one, 0..1 = none or one, * = many, 1..* = at least one. Many-to-many relations are split by a table: match_candidate links lost reports and found items, ownership_evidence links claims and item attributes.

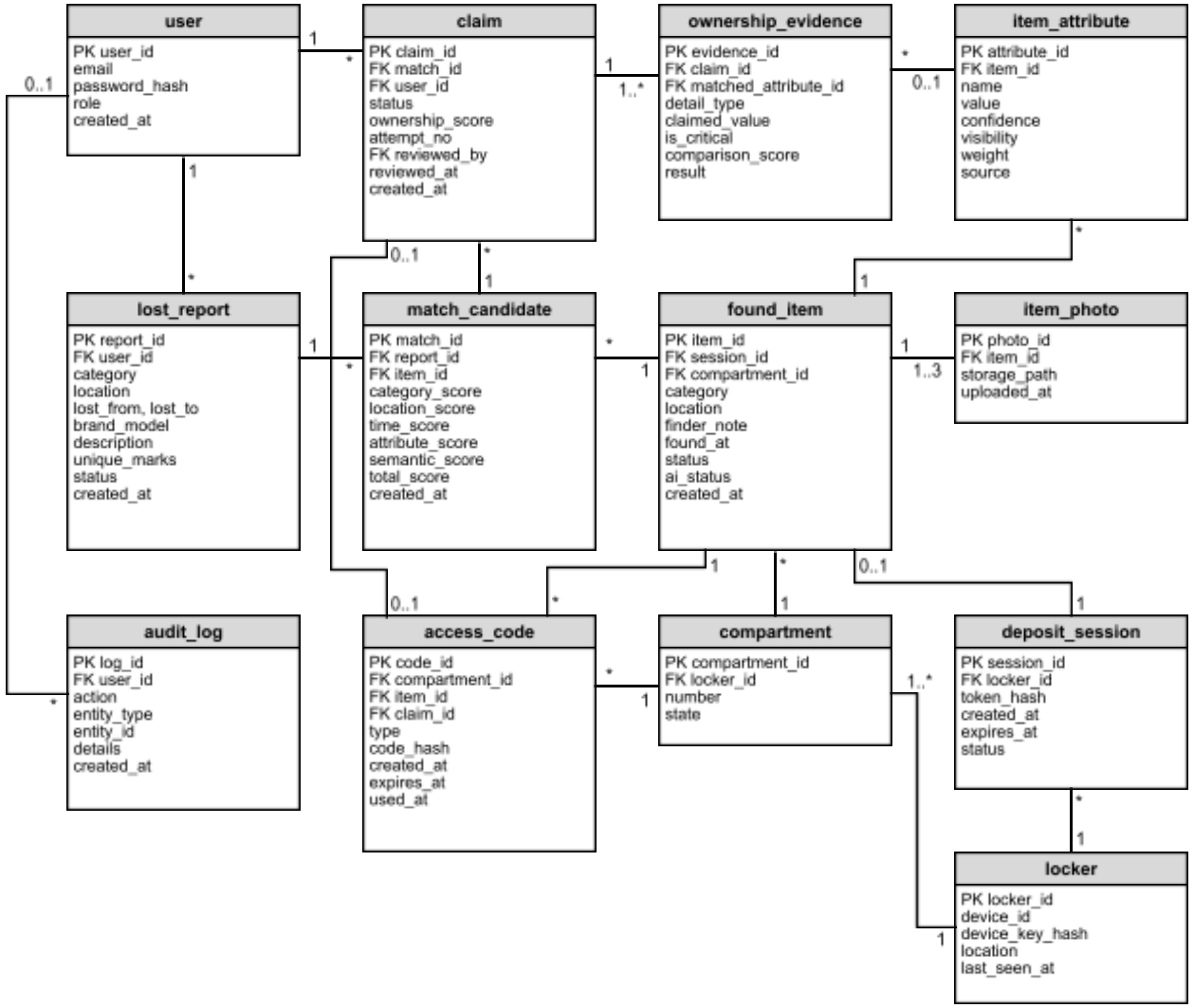


Figure 9: ER diagram of the PrivateFind database

- **Public / private:** item_attribute.visibility decides what candidates may see. Private attributes are only used for verification.
- **One owner per item:** the approval locks the found_item row and reserves the item in one transaction.
- **Codes:** access codes and the device key are stored as hashes with an expiry time.
- **Traceable score:** match_candidate stores the five partial scores and the total, so every Match Score can be explained.
- **Finder sessions:** deposit_session belongs to one locker and one found item, expires after a short time and stores the token only as a hash.
- **History:** a compartment keeps its old items as history, but only one active item is allowed at a time.

The MySQL schema and the first migration follow in Week 6.

9.5 Interfaces

All interfaces use JSON over HTTPS. The ESP32 sends its device key in every request.

Endpoint	Used by	Purpose
POST /api/deposits	Finder	Upload photos and note, get compartment and deposit code
POST /api/lost-reports	Owner	Create a lost-item report
GET /api/lost-reports/{id}/candidates	Owner	Top 3 candidates (public data only)
POST /api/claims	Owner	Submit ownership evidence, get decision
POST /api/admin/claims/{id}/decision	Admin	Approve or reject a claim
POST /api/device/codes/validate	ESP32	Check a keypad code, answer open or deny
POST /api/device/compartments/{no}/state	ESP32	Report if an item is inside

The AI service returns JSON with category, colours, brand, public attributes and private attributes, each with a confidence value. Pydantic checks the answer before it is stored.

9.6 User Interface

The pages are mobile-first, because finders and owners use their phone at the locker. Full photos are never shown to owners, only a short public summary.

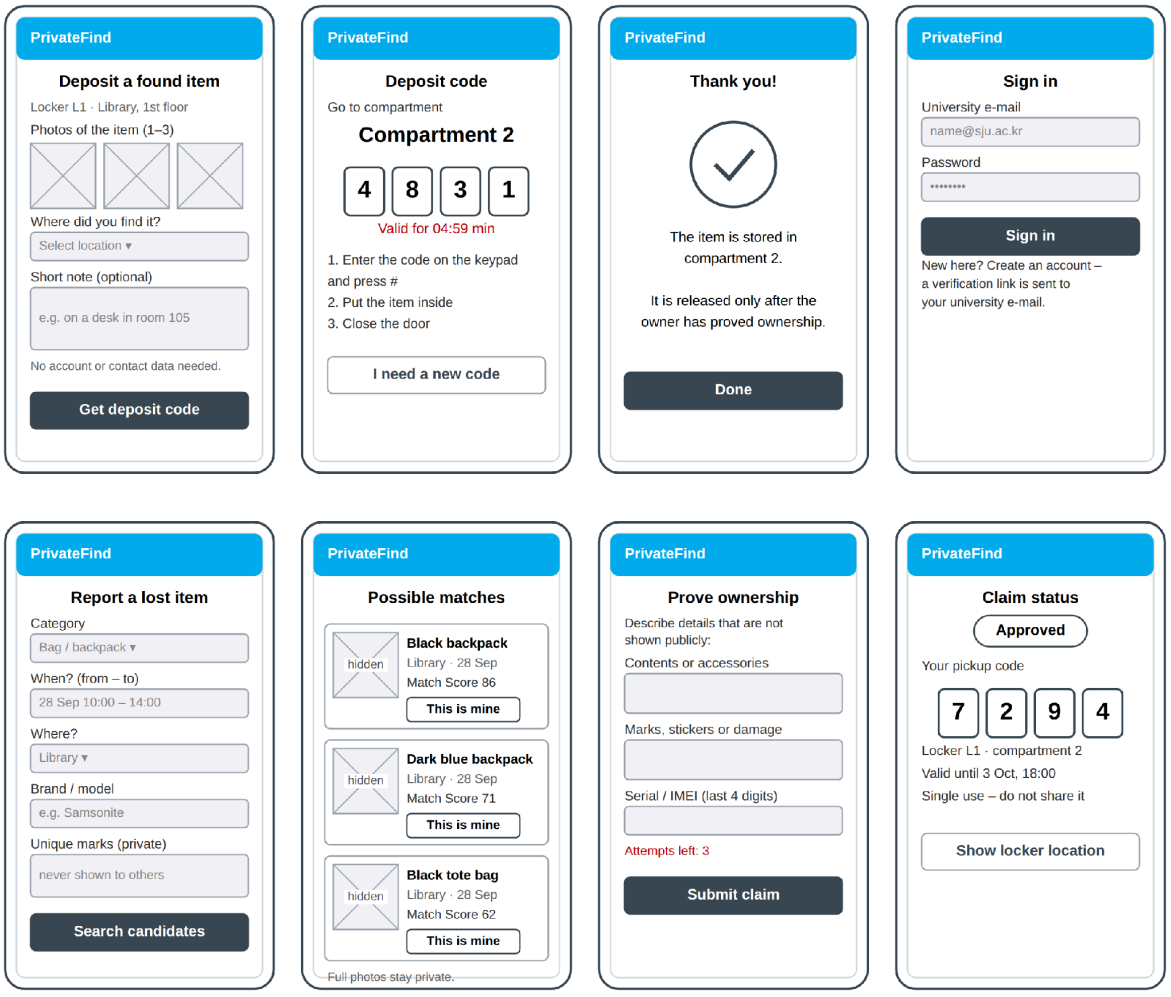


Figure 10: Wireframes of the finder flow (top) and the owner flow (bottom)

9.7 Locker Hardware

Component	ESP32 pin	Note
4x4 keypad	GPIO 13, 14, 27, 26 / 25, 33, 32, 4	Rows / columns
16x2 LCD (I2C)	GPIO 21 (SDA), 22 (SCL)	5 V supply
Servo compartment 1 / 2	GPIO 18 / 19	External 5 V, capacitor, common GND

Component	ESP32 pin	Note
IR sensor 1 / 2	GPIO 34 / 35	Input-only pins

The pin assignment above represents the planned final locker configuration. The current Week 5 breadboard prototype uses a temporary pin assignment for the keypad and first servo. The firmware will be remapped to the final assignment when the LCD, sensors and second compartment are integrated.

The ESP32, the keypad and the first servo already run on the breadboard (Figure 11). The other parts are ordered in Week 6.

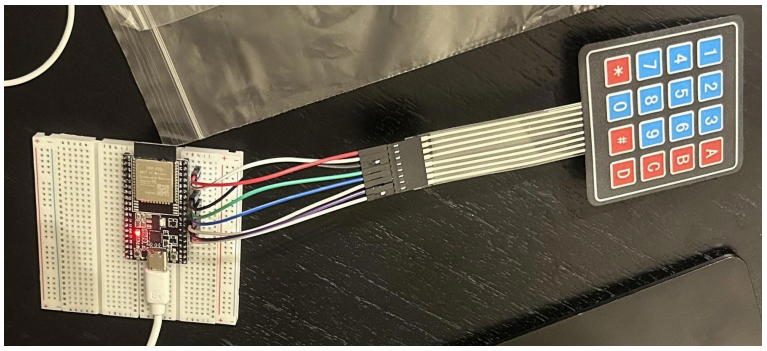


Figure 11: Breadboard test with ESP32 and keypad (Week 5)

10 References

Boehm, B. W. (1981). Software Engineering Economics. Prentice Hall.

Sommerville, I. Software Engineering, 10th edition. Pearson.

Sejong University (2026). Software Engineering Project Proposal Guidelines.

Sejong University (2026). Software Engineering Project Guide: Schedule, Themes and Tools.